

1. Architektura Projektu i Wprowadzenie

Opis: Dixit-AI to webowa implementacja gry Dixit z UI w przeglądarce, serwerem REST + real-time oraz oddzielnym serwisem AI/botów.

a. Stos technologiczny:

Frontend: React, TypeScript, Vite, Tailwind CSS, MUI, Radix UI, socket.io-client, zustand — sprawdź package.json.

Backend (API + real-time): Node.js, Express (v5), Socket.IO, Prisma (@prisma/client), PostgreSQL (pg), Redis (opcjonalnie), bcrypt, jsonwebtoken, multer — patrz package.json i [app.js](#).

AI / Bot: Python (>=3.12), FastAPI, uvicorn, google-genai, huggingface-hub, torch, sentence-transformers — patrz pyproject.toml.

Baza danych / ORM: PostgreSQL + Prisma — schema w schema.prisma.

Testy: Jest, supertest (backend).

Infra / Dev: Docker, docker-compose (docker-compose.yml, docker-compose.dev.yml), nodemon (dev backend), vite (dev frontend).

b. Git Flow:

Gałąź główna: main — chroniona, merge tylko przez PR.

Gałęzie robocze: feature/<nazwa>, fix/<nazwa>, hotfix/<nazwa>, opcjonalnie release/<ver>.

PR i review: Wszystkie zmiany przez Pull Request z co najmniej jednym review; wymagane zielone testy przed mergem.

Strategia merge'owania: squash merge lub rebase+merge (konsekwentnie w całym repo).

c. Struktura katalogów:

client: frontend React + Vite, UI, routing, socket client — (zob. package.json).

server: backend Express + Socket.IO, REST routes, logika gry, testy, prisma — (zob. app.js, package.json, schema.prisma).

bot: serwis AI/botów w Pythonie (FastAPI, modele/Generative APIs) — (zob. pyproject.toml).

src/: (root) dodatkowy kod aplikacji/komponenty UI (w repozytorium występuje src/app/...).

public: zasoby statyczne (obrazy kart, ikony).

docker-compose.yml, docker-compose.dev.yml, DOCKERFILE: konfiguracje konteneryzacji i uruchomienia.

changelog: notatki i historia zmian (np. CHANGELOG_SESSION2.md).

scripts: skrypty pomocnicze (seed, testy).

tests: testy jednostkowe i integracyjne (głównie w __tests__).

2. Baza Danych i Optymalizacja

a. Schemat Bazy Danych:

https://dbdiagram.io/d/dixit_ai-6a26a2c125fc5bf036b7655d

Opisy tabel:

users: Główna tabela przechowująca dane zarejestrowanych użytkowników.

rooms: Tabela zarządzająca pokojami gry, w których toczą się rozgrywki.

room_players: Tabela łącząca użytkowników (oraz boty) z konkretnymi pokojami, definiująca uczestników danej sesji.

games: Tabela reprezentująca pojedynczą sesję rozgrywki w ramach pokoju.

rounds: Tabela przechowująca informacje o poszczególnych rundach wewnątrz danej gry.

round_submissions: Tabela zawierająca zgłoszenia kart dokonane przez graczy w konkretnej rundzie.

round_votes: Tabela rejestrująca głosy oddane przez graczy na zgłoszone karty w danej rundzie.

cards: Główna tabela z zasobami kart dostępnymi w grze.

card_sets: Tabela grupująca karty w tematyczne zestawy.

unlockable_sets: Tabela definiująca zestawy kart, które można odblokować w grze po spełnieniu określonych warunków.

user_unlocks: Tabela przechowująca informację o tym, które zestawy zostały odblokowane przez konkretnego użytkownika.

user_stats: Tabela przechowująca zagregowane statystyki osiągnięć i zasoby (np. monety) użytkownika.

player_hands: Tabela odwzorowująca aktualną rękę kart przypisaną do gracza w trakcie rozgrywki.

player_card_history: Tabela przechowująca historię kart wykorzystanych przez gracza w danej grze.

game_scores: Tabela przechowująca końcowe lub bieżące wyniki oraz rankingi graczy w danej rozgrywce.

Główne relacje (skrót):

Użytkownik → Room (host): users.id → rooms.host_user_id (1:N).

Room → RoomPlayers: rooms.id → room_players.room_id (1:N);
room_players.user_id → users.id (0..1 — bots mają null).

Room → Games → Rounds: rooms.id → games.room_id (1:N), games.id → rounds.game_id (1:N).

Round → Submissions → Votes: rounds.id → round_submissions.round_id (1:N),
round_submissions.id → round_votes.voted_submission_id (1:N); każdy gracz
głosuje raz (round_id,voter_player_id unique).

CardSets → Cards: card_sets.id → cards.set_id (1:N).

Hands/History/Scores: player_hands, player_card_history, game_scores łączą
games ↔ room_players (i cards tam, gdzie potrzeba).

3. Logika Serwera i Multiplayer

a. Autoryzacja:

Mechanizm: JWT przechowywany w ciasteczku token (httpOnly). REST używa middleware auth — weryfikuje token z req.cookies i ustawia req.user (auth.js).

WebSocket: io.use w app.js parsuje socket.request.headers.cookie, weryfikuje JWT i ustawia socket.user; brak/nieprawidłowy token blokuje połączenie.

JWT payload: zawiera id i login (używane do identyfikacji użytkownika w socketach i REST).

Endpoints powiązane: POST /api/auth/login (ustawia cookie), POST /api/auth/logout (czyści cookie), GET /api/auth/me (zwraca req.user) — zob. auth.js.

Uwaga bezpieczeństwa: w produkcji stosować HTTPS i secure cookie, mocny process.env.JWT_SECRET i rozsądne czasy wygaśnięcia/refresh.

b. WebSockets ([Socket.io](#)):

Instancja & broadcast: Socket.io tworzony w app.js i przekazywany do REST przez socketBus.js (setIo/getIo) — pozwala RESTowi emitować zdarzenia do socketów (socketBus.js).

Konwencja rooms: lobby:CODE — lobby; gameId — sala gry; player:PLAYER_ID — prywatne notyfikacje dla jednego gracza.

Lobby flow: FE wywołuje REST POST /api/lobby/join → serwer dodaje room_player; następnie FE wywołuje socket join_room → lobbyHandler.js weryfikuje DB, dołącza do lobby:CODE i emituje lobbyUpdate.

Connect to game: event connect_to_game sprawdza, czy użytkownik jest uczestnikiem gry, ustawia socket.player_id i socket.game_id, dołącza do gameId i player:ID, wysyła connected_to_game z hand i stanem gry — implementacja w [gameHandler.js](#).

Główne eventy (client → server):

join_room: dołączenie do lobby (weryfikacja membership).

leave_room: opuszczenie lobby (host może zamknąć pokój).

connect_to_game: reconnect / join gry.

submit_prompt: narrator przesyła prompt + card_id — weryfikacja fazy i narratora; ustawia fazę submitting, tworzy round_submissions, uruchamia timer i triggeruje boty.

submit_card: gracz przesyła kartę — walidacja ręki i unikalności submission; jeśli wszyscy przesłali → przejście do voting.

submit_vote: walidacja (nie głosuj na swoją kartę, narrator nie głosuje, tylko raz); przy pełnym zbiorze głosów wywołuje zakończenie rundy.

disconnect_from_game: opuszczenie pokoi socketowych (nie usuwa DB entry).

Główne eventy (server → client): lobbyUpdate, game_started, connected_to_game, prompt_submitted, player_submitted_card, start_voting, player_voted, hand_updated (wysyłane do player:ID), round_ended, new_round, game_over.

Fazy i timery: serwer używa startPhaseTimer/clearPhaseTimer (wywoływane w miejscach takich jak submit_prompt, start gry) aby wymusić postęp faz (timeouty), a na wygaśnięcia przypisane są handlers z botOrchestrator (np. handleSubmittingExpiry, handleVotingExpiry). Zob. użycia w [server/handlers/gameHandler.js] i [server/routes/lobby.js].

Boty: botOrchestrator (asynchronicznie) wykonuje handleBotSubmissions, handleBotVotes, handleNarratorIfBot — generuje ruchy botów i może wyzwać zapisy w DB lub emitować eventy. Implementacja i integracja z serwisem AI znajduje się w botOrchestrator.js i oddzielnym serwisie bot.

Spójność danych: krytyczne operacje wykonują się w transakcjach Prisma (prisma.\$transaction) — np. tworzenie gry i rozdanie kart (POST /api/lobby/start w lobby.js), aktualizacja game_scores i przeniesienie kart do historii podczas zakończenia rundy (transakcja w _endRound w gameHandler.js). Dodatkowo serwer liczy wstawienia (count) po create aby uniknąć race conditions przy submission/vote.

Per-player updates: prywatne informacje (np. hand_updated) wysyłane są do player:PLAYER_ID, nie broadcastowane do całej gry.

Obsługa obrazków: karty zwracają `image_url` jako endpoint REST (`/api/cards/:id/image`) — front używa tego URL do pobrania obrazu.

Krótki sekwencyjny flow gry

1. Użytkownik loguje się (POST `/api/auth/login`) → cookie token.
2. Tworzy/dołącza do lobby (POST `/api/lobby/create` / POST `/api/lobby/join`), FE otwiera socket i wywołuje `join_room`.
3. Host POST `/api/lobby/start` → serwer tworzy `game`, tworzy `game_scores`, rozdziela `player_hands`, tworzy rundę 1.
4. Gracze łączą się socketem `connect_to_game` i otrzymują `connected_to_game` z ręką.
5. Narrator `submit_prompt` → faza `submitting` + timer + bot submissions.
6. Gracze `submit_card` → kiedy wszyscy przesłali → `start_voting`.
7. Gracze `submit_vote` → kiedy wszystkie głosy (lub timer) → `_endRound`: policz punkty, zaktualizuj `game_scores` (transakcja), przenieś karty do historii, rozdaj nowe karty, emituj `round_ended` i `new_round` lub `game_over`.

4. Interfejs Użytkownika - Frontend

a. Routing i Stan:

Mechanizm: React Router z `createBrowserRouter` — główny router w `routes.tsx`.

Układ: wszystkie trasy renderowane w `MainLayout` (`MainLayout.tsx`).

Trasy: `/` (Auth), `/menu`, `/personalization`, `/stats`, `/psycho-profile`, `/host`, `/join`, `/game`, `/my-cards` — część zabezpieczona wrapperami `RequireAuth` / `GuestOnly` (`routes.tsx`).

Auto-nawigacja do gry: logika przekierowania na `/game` oparta o fazę gry w store (`gameNavigation.ts`) — `gameNavigation.ts`.

Provider: `RouterProvider` użyty w `App.tsx`.

Stan aplikacji

Store: Zustand useGameStore centralizuje stan gry — pola: user, roomCode, gameld, currentPhase, players, myHand, tableCards, narratorPrompt, timer, myId, plus akcje (setGameState, setMyHand, resetGame) — useGameStore.ts.

Mapowanie eventów → stan: socket handlers rejestrują eventy serwera i aktualizują store (connected_to_game, prompt_submitted, start_voting, hand_updated, round_ended, new_round, timerTick, gameStateUpdate) — gameSocket.ts.

Sesja: restoreSessionFromCookie() pobiera /api/auth/me, ustawia profil i łączy socket — session.ts.

Socket init: połączenie i globalne listener-y w socket.ts.

b. Użyte komponenty (wybrane):

Layout / Nawigacja: MainLayout, ViewPageShell, GameNavigationBridge — components.

Autoryzacja / Widoki: RequireAuth, GuestOnly, AuthView, MainMenuView, HostGameView, JoinLobbyView — views.

Gameplay: GameBoard (główna plansza), GameCard, GameplayComponents, CardCanvasEditor — (zob. client/src/app/views/Gameplay/GameBoard).

Lobby / UI pomocnicze: LobbyPlayerList, ApiStateMessage, Button, Input, MenuHelpFooter, PlayfulBackground.

UI primitives: zestaw helperów/wrapperów w components/ui/* (Radix / MUI wrappers).

Usługi / komunikacja

REST API client: api.ts, lobbyApi.ts, personalizationApi.ts, psychoProfileApi.ts — wywołania do backendu i mapowania DTO.

Sockety: socket.ts (init), lobbySocket.ts (join/leave lobby), gameSocket.ts (emity i handlery) — services.

Mapowanie obrazków: klient normalizuje URL kart (endpoint /api/cards/:id/image) w mapServerCard.

Najważniejsze biblioteki

React, React Router, TypeScript, Vite, Tailwind CSS, MUI, Radix UI, Zustand (store), socket.io-client.

5. Sztuczna Inteligencja i NLP

a. Generowanie tagów:

Celem komponentu jest automatyczne przypisywanie tagów do obrazów kart (`cards.tags`), co umożliwi filtrowanie, wyszukiwanie oraz lepszą kategoryzację zasobów w bazie danych.

Wymagania dla tagów:

- **Format:** Zwięzły zbiór słów/fraz (3–8 elementów).
- **Język:** Domyślnie PL (opcjonalnie EN).
- **Jakość:** Wysoka trafność, de-duplikacja, brak redundancji (normalizacja).

Proces generowania tagów opiera się na hybrydowym podejściu, łączącym szybkość metod embeddingowych z precyzją modeli LLM.

1. **Ekstrakcja cech:** Wykorzystanie istniejących modeli typu CLIP (np. `clip-ViT-B-32-multilingual`) do stworzenia wektora reprezentacji obrazu.
2. **Kandydowanie:** Porównanie wektora obrazu z wektorami przygotowanego słownika tagów (similarity scoring) w celu wyłonienia top-N kandydatów.
3. **Normalizacja (LLM):** Przekazanie kandydatów do modelu LLM (Gemini) w celu usunięcia synonimów, de-duplikacji oraz upewnienia się, że tagi są zrozumiałe dla użytkownika (format wyjściowy: JSON/CSV).
4. **Wzbogacenie (Captioning - opcjonalnie):** W razie niskiej trafności, zastosowanie modelu typu BLIP2/OFA do wygenerowania surowego opisu (caption), z którego ekstrakcja rzeczowników stanowi dodatkowe źródło tagów.
5. **Walidacja:** Usuwanie *stop-words*, weryfikacja pod kątem polityk bezpieczeństwa oraz odrzucanie tagów o niskim wskaźniku zaufania.

Serwer wystawia endpoint do wyzwalania procesu generowania tagów dla konkretnej karty:

- **Endpoint:** `POST /api/cards/:id/generate-tags`
- **Logika:** * Wymagana autoryzacja.
 - Obsługa parametru `refresh` (boolean) pozwalająca na nadpisanie istniejących tagów.

- Status **202 Accepted** w przypadku przetwarzania asynchronicznego (rekomendowane w celu uniknięcia timeoutów przy użyciu LLM).

Bot Service

- **Endpoint:** `POST /generate-tags/`
- **Request:** `{ image_id: string }` lub `{ image_b64: string }`
- **Response:** `{ tags: string[] }`
- **Implementacja:** Dodanie nowej ścieżki w `bot/src/main.py` oraz logiki biznesowej w nowym module `bot/src/taggers.py`.

Wybór narzędzi i optymalizacja

- **Modele:** *Embeddings:*
`sentence-transformers/clip-ViT-B-32-multilingual-v1`.
 - *LLM Normalizacja:* Google Gemini (reuse istniejącej integracji).
- **Optymalizacja kosztów:**
 - LLM używany wyłącznie do finalnej normalizacji (post-processing), a nie do klasyfikacji obrazu.
 - Wprowadzenie cache'owania wyników dla identycznych obrazów (hashowanie obrazu).
- **Asynchroniczność:** Generowanie tagów powinno być procesem wykonywanym w tle (background job), aby nie blokować głównego wątku serwera podczas uploadu karty przez użytkownika.

Bezpieczeństwo i moderacja

- Każdy wygenerowany tag podlega filtrowaniu pod kątem słów niedozwolonych (NSFW).
- Automatyczne flagowanie rekordów, dla których model uzyskał niski wynik pewności (`confidence score`).

6. Game Design i UX

a. Zasady gry:

Ogólny opis:

Gra dla minimum 4 osób na rozgrywkę, max 6. W każdej rozgrywce jest opcja dodania botów AI. Nie ma opcji rozpoczęcia gry, póki nie uczestniczy w niej min. 4 graczy i/lub botów AI.

Przebieg rundy:

W pierwszej rundzie gry narratorem zostaje losowo wybrany gracz.

1. NARRATOR. Jeden z graczy zostaje narratorem. Narrator wybiera 1 z 6 kart należących do niego. Narrator promptuje jakiekolwiek skojarzenie związane z treścią wybranej karty (max 60 znaków - warto wziąć pod uwagę zapisywanie danych słów wielkimi literami lub używanie emotikon). Skojarzenie nie może być zbyt oczywiste – wtedy narrator nie zdobędzie punktów w rundzie. Narrator ma na to max **40 sek.**

W wypadku gdy narrator nie udzieli odpowiedzi (pusty prompt - nawet jeśli wybierze kartę) - następuje przejście do kolejnej rundy. Wyświetlany jest ekran punktacji na 5 sek.

2. GRACZE. Każdy z pozostałych graczy wybiera 1 z 6 swoich kart. Każdy ma na to **30 sek.** Jego zdaniem jest wybranie ilustracji, która najlepiej obrazuje prompt narratora.

Po upływie czasu wybierana jest losowa karta z decku gracza.

3. GŁOSOWANIE GRACZY. Wszystkie karty, które zostały wybrane w tej rundzie są wyświetlane w losowej kolejności. Celem wszystkich pozostałych graczy jest odnalezienie wśród wyświetlonych kart tej, której właścicielem jest narrator. Gracze mają **20 sek.** na oddanie swojego głosu - wybranie karty należącej do narratora.

Istnieje szansa, że gracz nie wskaże żadnej karty - gracz nie otrzymuje punktów bonusowych, gdy jego karta zostanie wskazana przez innego gracza w fazie głosowania.

Zasady przyznawania punktów po każdej rundzie (też poniżej):

Jeśli wszyscy gracze uczestniczący w głosowaniu poprawnie wskazali kartę należącą do narratora albo gdy żaden z graczy jej nie wskazał:

- Narrator nie otrzymuje żadnych punktów.
- Wszyscy pozostali gracze zdobywają po 2.

Jeśli tylko niektórzy gracze zagłosowali na kartę narratora:

- Narrator zdobywa 3 pkt.

- Gracze, którzy poprawnie wskazali kartę narratora, także zdobywają 3 pkt. Pozostali gracze nie otrzymują żadnych punktów.

Dodatkowo każdy gracz (z wyjątkiem narratora i osoby która nie wybrała karty w fazie głosowania graczy), zdobywa 1 bonusowy punkt za każdy głos oddany na jego kartę.

4. PUNKTACJA. Punkty są sumowane z zdobytymi w poprzednich rundach i wyświetlane. Do ręki każdego gracza dobierana jest 1 karta, aby było ich w sumie 6. Nowym narratorem zostaje kolejny gracz.

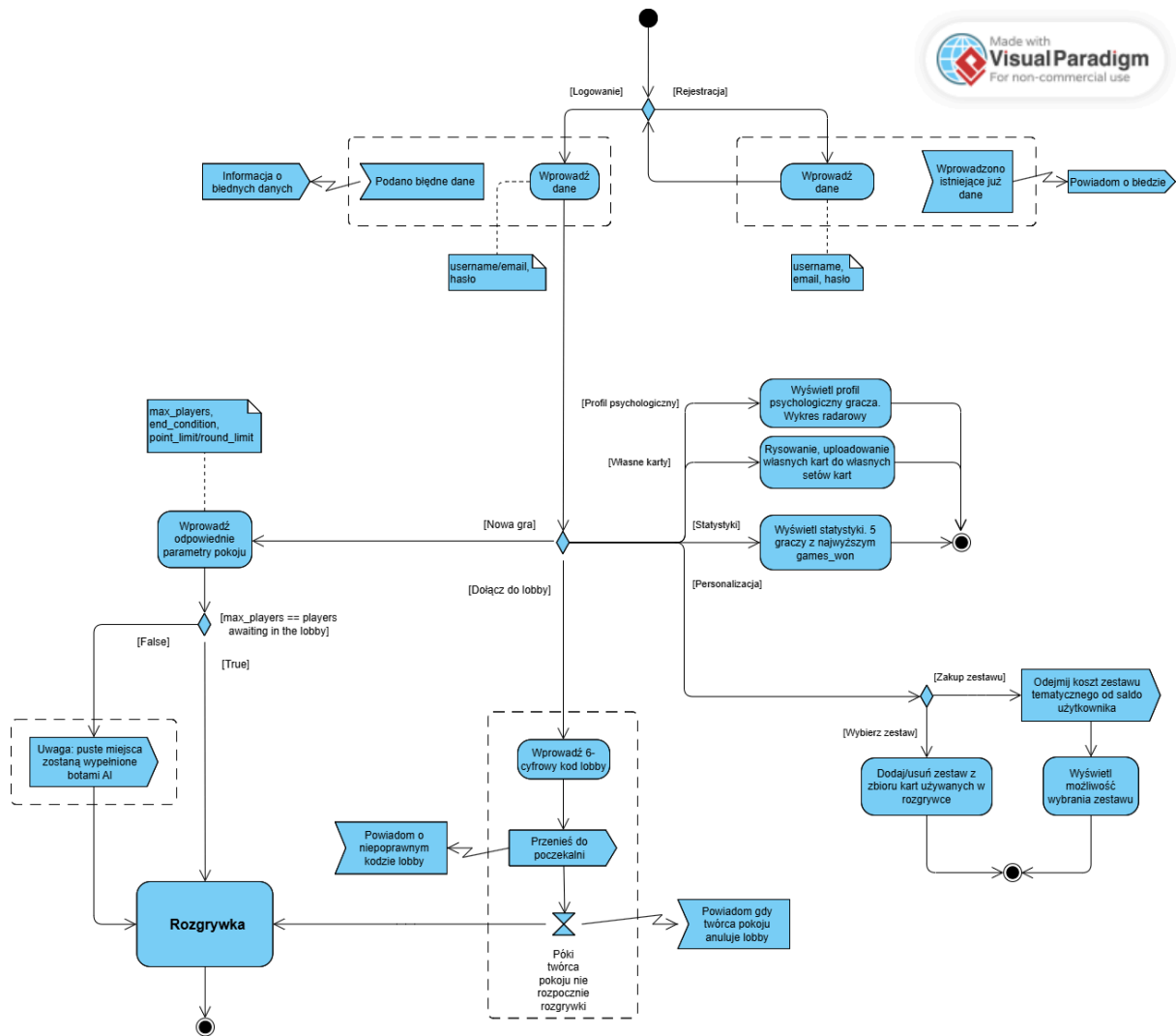
*Gra kończy się po rundzie, w której jeden z graczy zdobył co najmniej 30 punktów (klasyczna wersja gry) lub gracz z największą liczbą punktów po upływie zadanej przed rozgrywką liczbie rund (*min 2*). W przypadku remisu remisujący gracze współdzielą zwycięstwo.*

	narrator	gracz który zgadł	gracz który nie zgadł	gracz na którego kartę został oddany głos*
wszyscy zgadli	0 pkt	2 pkt	-	-
nikt nie zgadł	0 pkt	-	2 pkt	1 pkt / głos
część zgadła	3 pkt	3 pkt	0 pkt	1 pkt / głos

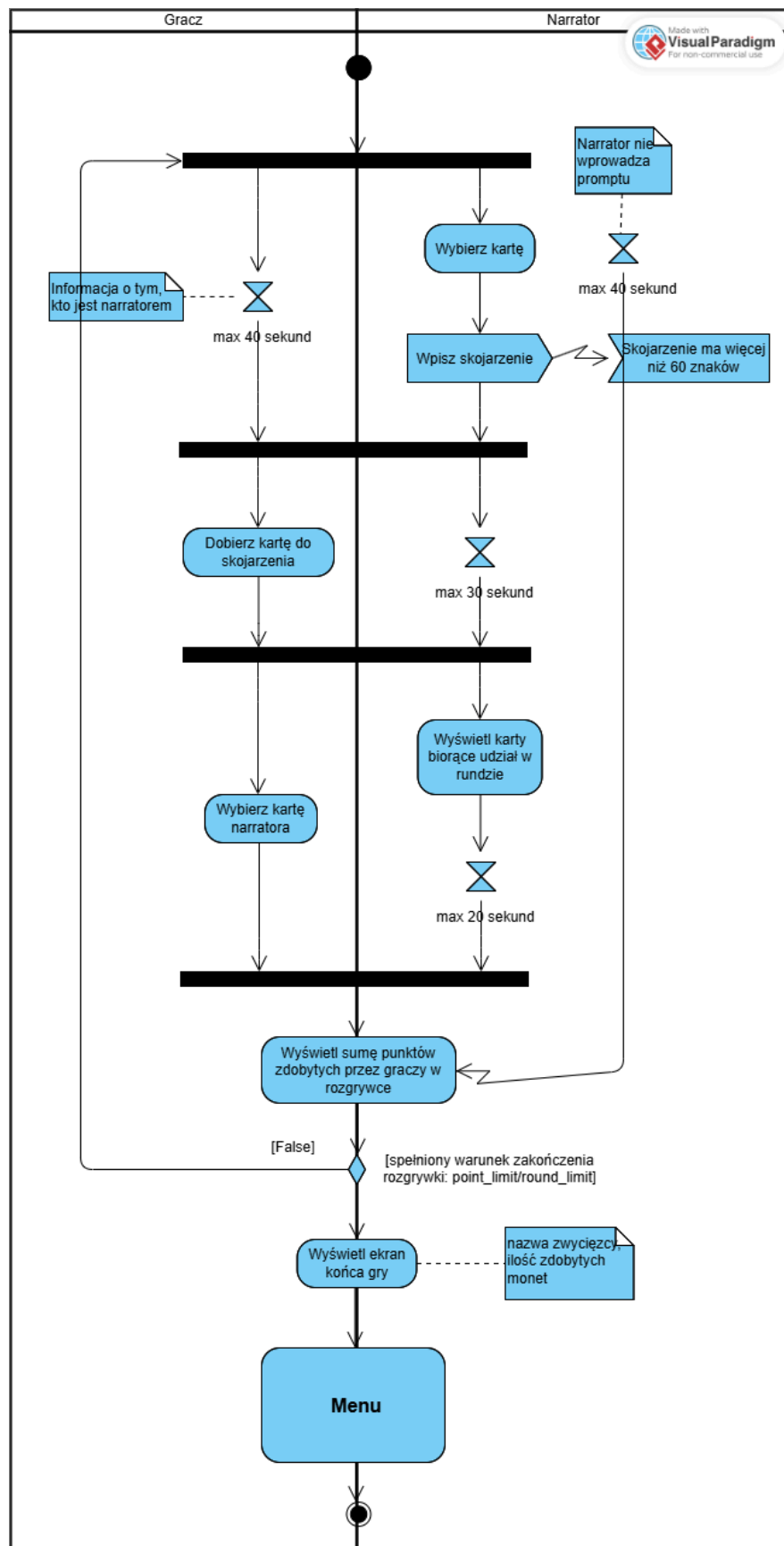
* 0pkt / głos jeśli gracz nie wybrał żadnej karty w fazie głosowania graczy.

b. **UX Flow:**

Made with **Visual Paradigm**
For non-commercial use



Flow Rozgrywka:



7. Grafika i Wizualia

a. Standardy Assetów:

Struktura katalogów:

- `server/public/Karty/` – statyczne zasoby kart (backend).
- `client/public/Ikony/` – zasoby UI (wektorowe).

Standardy kart (Face, Card Back)

Rozdzielczości:

- **Baseline (1x):** 1024 × 1365 px.
- **High-Res (2x):** 2048 × 2730 px.

Formaty plików: PNG-24. Format preferowany

Limity: Maksymalny rozmiar pliku wynosi **2 MB** (ograniczenie `multer`).

Proporcje: Pionowe, stosunek 3:4.

Ikony UI

- **Format:** SVG.
- **Wymiary:** Standardowy `viewBox` 24×24 px lub 20×20 px.

Endpoints:

- `GET /api/cards/:id/image`: Obsługa serwowania danych binarnych lub plików statycznych.
- `POST /api/cards/upload`: Obsługa przesłanych plików z walidacją `fileSize` (2 MB) oraz MIME types (`image/png`, `image/webp`).

8. Quality Assurance i Deployment

a. Środowisko

Quality Assurance & Deployment

Cel: zapewnić stabilne, powtarzalne wydania; testy przed mergem; łatwy rollback i monitoring produkcji.

Proces wydania: feature → PR → CI (lint/test/build) → merge → staging → smoke tests → ręczne/automatyczne wdrożenie na production; ochrona gałęzi main i wymóg review.

CI (zalecane kroki): install deps, lint, unit tests, integration tests, build artefaktów, budowa obrazu kontenera, test migracji, security & dependency scan.

Budowa i wdrożenie: obrazy Docker (tagowanie semver), push do registry, uruchomienie z docker-compose lub orchestratorami (K8s). Przykład lokalnego buildu:

```
docker-compose -f docker-compose.yml up --build -d
```

Migracje DB: uruchamiaj migracje podczas deployu: npx prisma migrate deploy (production) / npx prisma migrate dev (dev).

Środowisko

Pliki konfiguracyjne: konfiguracja docker/compose w repo: docker-compose.dev.yml i docker-compose.yml, obrazy/Build: DOCKERFILE. Bot: pyproject.toml.

Lokalny start (przykład):

```
# cały stack deweloperski
docker-compose -f docker-compose.dev.yml up --build

# lub ręcznie: serwer i klient
cd server && npm ci && npm run dev
cd client && npm ci && npm run dev
```

Wymagane usługi: PostgreSQL, Redis (opcjonalnie), serwis botów (Python), storage (S3 lub lokalne), reverse proxy/SSL w prod.

Zmienne środowiskowe: DATABASE_URL, JWT_SECRET, COOKIE_NAME/COOKIE_SECRET, REDIS_URL, PORT, GOOGLE_API_KEY / BOT_API_KEY, S3_BUCKET — przechowuj w CI secrets/vault.

Migracje i seed: przed wdrożeniem/stagingiem uruchom migracje i (opcjonalnie) seed: npx prisma migrate deploy / npm run seed.

9. Instrukcja dla Użytkownika / Oceniającego

a. Jak odpalić grę (Docker).

```
docker-compose up --build
```

Lub lokalnie:

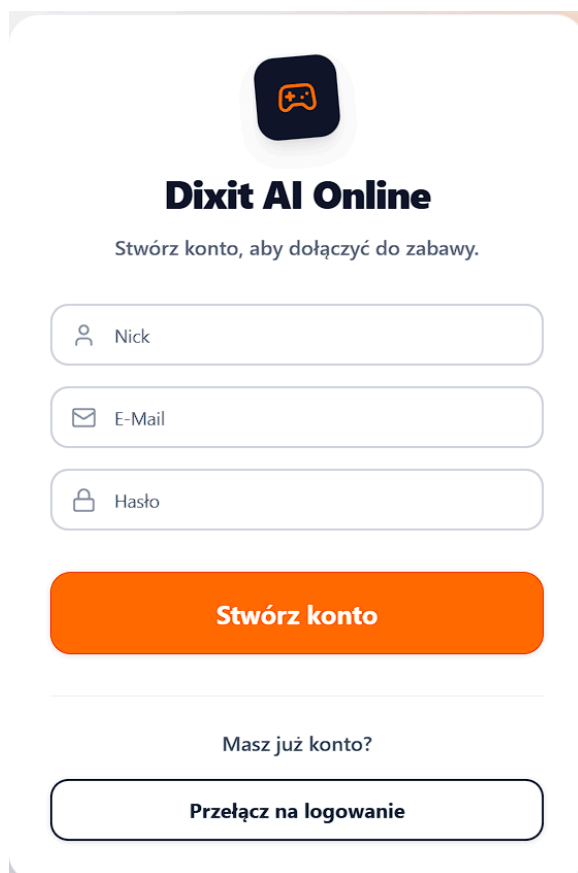
Frontend:

```
cd client  
npm install  
npm run dev
```

Backend:

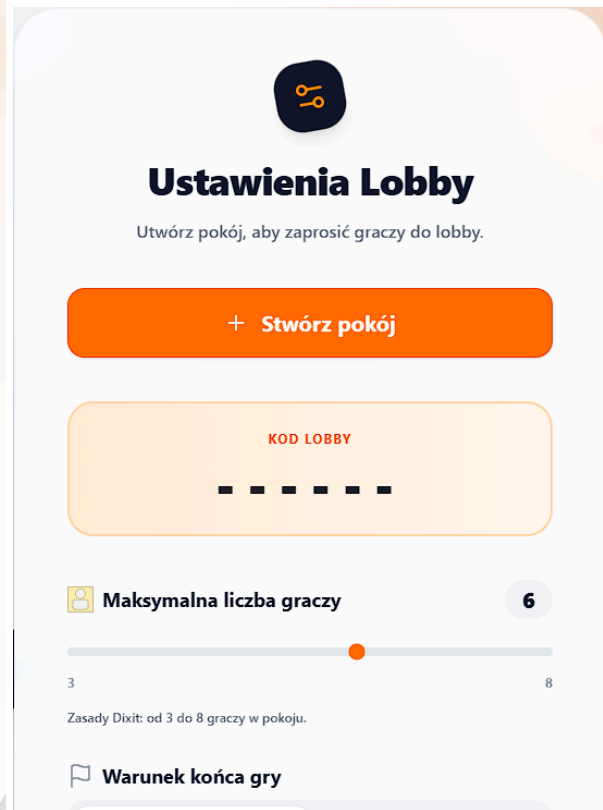
```
cd server  
npm install  
npm run dev
```

b. Jak założyć konto.



The screenshot shows a registration form for 'Dixit AI Online'. At the top is a logo featuring a game controller icon. Below the logo is the title 'Dixit AI Online' and a subtitle 'Stwórz konto, aby dołączyć do zabawy.' (Create account to join the game). The form consists of three input fields: 'Nick' with a person icon, 'E-Mail' with an envelope icon, and 'Hasło' (Password) with a lock icon. Below these fields is a large orange button labeled 'Stwórz konto'. A horizontal line separates this section from the login section below. The login section has the text 'Masz już konto?' (Do you already have an account?) and a button labeled 'Przełącz na logowanie' (Switch to login).

c. Jak stworzyć stół.



Stwórz nową grę -> + Stwórz pokój